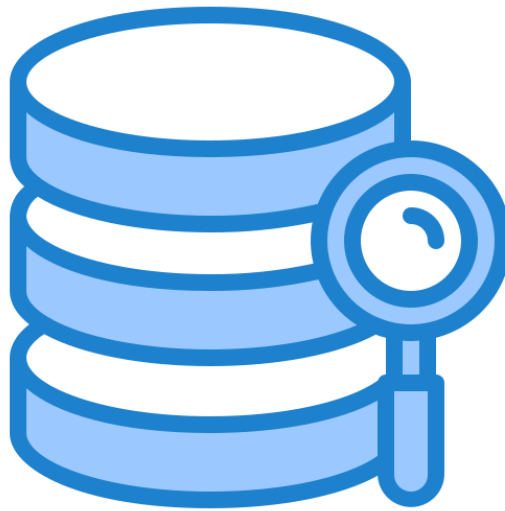


Projeto de Bases de Dados “VINISYS”

PARTE-II



**Ano letivo 2024/2025
1º Semestre**

**Docentes (PL)
- Bráulio Alturas
- Joaquim Lourenço dos Santos Esmerado**

Grupo 7

- Francisco Martins Ribeiro – Aluno N.º 123930
- Pedro Miguel Pereira Clemente – Aluno N.º 110337
- Rodrigo Miguel da Silva Delaunay - Aluno N.º 122123
- Rúben Manuel Henriques Rocha – Aluno N.º 66174

Dezembro, 2024

Índice

Introdução	1
Automatismos	2
Pesquisa de Dados	6
Sistema <i>Web-based</i>	11
Conclusão.....	12

Introdução

O presente relatório, elaborado no âmbito da Unidade Curricular (UC) de Bases de Dados, constitui a Parte II do projeto "VINISYS". Nesta fase, o objetivo delineado reside no desenvolvimento de funcionalidades adicionais que complementam a estrutura base já implementada, com vistas a tornar o sistema de informação mais dinâmico, eficiente e acessível.

A segunda parte do projeto engloba três pilares fundamentais que serão abordados no presente relatório: (i) automatismos, (ii) pesquisa de dados e a (iii) implementação de um sistema web-based. Esses elementos visam não só otimizar os processos de gestão, mas também aprimorar a experiência do utilizador e facilitar a administração centralizada dos dados relacionados à produção, armazenamento e comercialização de vinhos.

Neste relatório, serão detalhados os seguintes tópicos:

1. Automatismos - Implementação de *triggers*, *stored procedures* e funções que automatizam tarefas essenciais, garantindo maior eficiência e consistência na gestão da base de dados.
2. Pesquisa de Dados - Criação de comandos SQL e *views* que respondem a diversas necessidades analíticas, oferecendo informações estruturadas e insights para a gestão da empresa.
3. Sistema Web-Based - Desenvolvimento de um protótipo em HTML/PHP que demonstra funcionalidades como o registo de colheitas, listagem de vinhos e gestão de vendas, contribuindo para uma interface intuitiva e operacional.

Este documento busca detalhar cada uma dessas etapas, demonstrando como as soluções propostas atendem aos requisitos estabelecidos e contribuem para a excelência do sistema "VINISYS".

Automatismos

Conforme proposto no primeiro ponto do enunciado da Parte II do projeto "VINISYS", é solicitada a elaboração de automatizações com o objetivo de otimizar a base de dados do sistema de informação e gerir a distribuição de vinhos ao utilizar *triggers*, *stored procedures* e funções.

Descreve-se, de seguida, a resposta individual a cada um dos automatismos solicitados no enunciado.

1.1. Triggers

1.1.1. “T 1. Atualizar a quantidade cultivada em PRODUZ ao inserir um novo registo de plantação de uma casta em uma vinha.”

Importa destacar uma limitação que se identificou relativamente à implementação deste *trigger*. Devido ao facto de VINHA_codigo e CASTA_ID constituírem a chave primária composta da tabela PRODUZ, não é possível efetuar um comando INSERT efetivo pois, naturalmente, o SQL não permite a repetição de chaves primárias na mesma tabela. A solução poderá passar por duas opções, que se descrevem de seguida:

- a. Criação de uma tabela temporária (onde se armazenem os dados que se pretendem inserir), verificação da tabela PRODUZ caso haja repetição de VINHA_codigo e CASTA_ID e, posteriormente, efetuar UPDATE da coluna área_cultivada na tabela PRODUZ;
- b. Ao utilizar o comando INSERT INTO, adicionar “ON DUPLICATE KEY UPDATE área_cultivada = área_cultivada + VALUES(área_cultivada);”. Esta solução, apesar de viável, invalida a utilização do *trigger* que se pretende elaborar.

Atentos ao facto de nenhuma das soluções anteriormente apresentadas preveem a elaboração de um *trigger* que atue somente na tabela PRODUZ, acabou por optar-se por uma terceira opção, que prevê a implementação de um *trigger* que efetue o UPDATE da coluna área_cultivada e não execute o INSERT INTO (prevenindo, desta forma, o erro relacionado com chaves primárias duplicadas):

```
CREATE TRIGGER T1
BEFORE INSERT ON PRODUZ
FOR EACH ROW
BEGIN
    DECLARE existing_area INT;
```

```

SELECT area_cultivada INTO existing_area
FROM PRODUZ
WHERE VINHA_codigo = NEW.VINHA_codigo AND CASTA_ID =
NEW.CASTA_ID
LIMIT 1;
IF existing_area IS NOT NULL THEN
    UPDATE PRODUZ
    SET area_cultivada = existing_area + NEW.area_cultivada
    WHERE VINHA_codigo = NEW.VINHA_codigo AND CASTA_ID =
NEW.CASTA_ID;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Já existe registo na
Base de Dados. Área cultivada atualizada.';
END IF;
END

```

- 1.1.2. “T 2. Registrar automaticamente a data de início de uma colheita na tabela COLHEITA ao inserir o primeiro trabalhador em PARTICIPA”

```

CREATE TRIGGER T2
AFTER INSERT ON PARTICIPA
FOR EACH ROW
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM COLHEITA WHERE COLHEITA.COLHEITA_ID =
NEW.COLHEITA_ID AND data_inicio IS NOT NULL
    ) THEN
        UPDATE COLHEITA
        SET data_inicio = CURRENT_DATE
        WHERE COLHEITA_ID = NEW.COLHEITA_ID;
    END IF;
END;

```

1.2. *Stored Procedures*

- 1.2.1. “P 1. Registrar uma nova participação de um trabalhador em uma colheita e calcular a remuneração total.”

```
DELIMITER $$  
CREATE PROCEDURE P1(  
    IN colheita_id INT,  
    IN trabalhador_id INT,  
    IN horas INT,  
    IN remuneracao_por_hora DECIMAL(10, 2),  
    OUT remuneracao_total DECIMAL(10, 2)  
)  
BEGIN  
    INSERT INTO PARTICIPA (COLHEITA_ID,  
TRABALHADOR_COLABORADOR_numero, horas, remuneracao)  
VALUES (colheita_id, trabalhador_id, horas, horas * remuneracao_por_hora);  
SELECT SUM(remuneracao) INTO remuneracao_total  
FROM PARTICIPA  
WHERE COLHEITA_ID = colheita_id;  
END$$  
DELIMITER;
```

- 1.2.2. “P 2. Registrar uma nova colheita e associar um produtor e uma região.”

```
DELIMITER $$  
CREATE PROCEDURE P2(  
    IN ano_colheita INT,  
    IN regioao_id INT,  
    IN produtor_id INT,  
    OUT colheita_id INT  
)  
BEGIN  
    INSERT INTO COLHEITA (ano, REGIAO_ID, PRODUTOR_ID)  
VALUES (ano_colheita, regioao_id, produtor_id);  
SET colheita_id = LAST_INSERT_ID();
```

```
END$$  
DELIMITER ;
```

1.3. Funções

1.3.1. “F 1. Crie uma função para calcular o total de horas trabalhadas em uma colheita específica.”

```
CREATE FUNCTION F1(colheita_id INT)  
RETURNS INT  
DETERMINISTIC  
BEGIN  
    DECLARE total_horas INT;  
    SELECT SUM(horas) INTO total_horas  
    FROM PARTICIPA  
    WHERE participa.COLHEITA_ID = colheita_id;  
    RETURN total_horas;  
END;
```

1.3.2. “F 2. Crie uma função para calcular a área total cultivada de uma casta específica em todas as vinhas.”

```
CREATE FUNCTION F2(casta_id INT)  
RETURNS INT  
DETERMINISTIC  
BEGIN  
    DECLARE area_total INT;  
    SELECT SUM(area_cultivada) INTO area_total  
    FROM PRODUZ  
    WHERE produz.CASTA_ID = casta_id;  
    RETURN area_total;  
END;
```

Pesquisa de Dados

É proposto no segundo ponto do enunciado da Parte II do projeto "VINISYS" a elaboração de comandos *select* por forma a criar *views* pertinentes à gestão da informação disponível na Base de Dados elaborada.

Descreve-se, de seguida, a resposta individual a cada um dos comandos *select* solicitados no enunciado.

1. “Q1. Apresente todas as vinhas e respetivas castas ordenadas por vinha e casta.”

```
SELECT vinha.nome_vinha AS Vinha, casta.tipo_uva AS Casta
FROM vinha
JOIN produz ON vinha.codigo = produz.VINHA_codigo
JOIN casta ON produz.CASTA_ID = casta.CASTA_ID
ORDER BY vinha.nome_vinha, casta.tipo_uva;
```

2. “Q2. Apresente os três vinhos mais premiados.”

```
SELECT v.nome AS Vinho, COUNT(p.PREMIO_ID) AS Prémios
FROM vinho v
JOIN edicao e ON v.VINHO_ID = e.VINHO_ID
JOIN ganha g ON e.EDICAO_ID = g.EDICAO_ID
JOIN premio p ON p.PREMIO_ID = g.PREMIO_ID
GROUP BY v.VINHO_ID
ORDER BY Prémios DESC
LIMIT 3;
```

3. “Q3. Apresente os vinhos que receberam prémios consecutivos em dois ou mais anos seguidos.”

```
SELECT DISTINCT v.nome AS Vinho
FROM vinho v
JOIN edicao e ON v.VINHO_ID = e.VINHO_ID
JOIN ganha g ON e.EDICAO_ID = g.EDICAO_ID
JOIN premio p ON g.PREMIO_ID = p.PREMIO_ID
WHERE EXISTS (
    SELECT 1
    FROM ganha g1
    JOIN edicao e1 ON g1.EDICAO_ID = e1.EDICAO_ID
```



```

        JOIN premio p1 ON g1.PREMIO_ID = p1.PREMIO_ID
        WHERE e1.VINHO_ID = v.VINHO_ID
        AND YEAR(g1.data_distincao) = YEAR(g.data_distincao) + 1
    )
ORDER BY Vinho;

```

4. “Q4. Apresente as castas mais produzidas em cada região.”

```

SELECT
    r.denominacao AS regioao,
    c.tipo_uva AS casta,
    SUM(co.quantidade) AS quantidade_produzida
FROM
    regioao r
JOIN
    colheita co ON r.REGIAO_ID = co.REGIAO_ID
JOIN
    produz p ON p.VINHA_codigo = co.PRODUTOR_ID
JOIN
    casta c ON p.CASTA_ID = c.CASTA_ID
GROUP BY
    r.denominacao, c.tipo_uva
ORDER BY
    quantidade_produzida DESC
LIMIT 0, 25;

```

5. “Q5. Apresente os vinhos com valor total de devoluções superior a 100€ em 2023.”

```
SELECT
    vinho,
    total_devolucoes
FROM
    (
        SELECT
            v.VINHO_ID,
            v.nome AS vinho,
            SUM(dev.montante) AS total_devolucoes
        FROM
            devolucao dev
        JOIN
            devolvido dv ON dev.DEVOLUCAO_ID = dv.DEVOLUCAO_ID
        JOIN
            vinho v ON dv.VINHO_ID = v.VINHO_ID
        JOIN
            item i ON i.VINHO_ID = v.VINHO_ID
        JOIN
            fatura f ON i.FATURA_num_fatura = f.num_fatura
        WHERE
            YEAR(dev.data_devol) = 2023
        GROUP BY
            v.VINHO_ID
    ) AS vinhos_com_devolucoes
WHERE
    total_devolucoes > 100;
```

6. “Q 6. Apresente os vinhos mais vendidos por cada produtor.”

```
SELECT
    p.nome_vinicola AS produtor,
    v.nome AS vinho,
    i.quantidade AS quantidade_vendida
FROM
    produtor p
JOIN
    vinho v ON p.PRODUTOR_ID = v.PRODUTOR_ID
JOIN
    item i ON v.VINHO_ID = i.VINHO_ID
WHERE
    i.quantidade = (
        SELECT MAX(i2.quantidade)
        FROM item i2
        WHERE i2.VINHO_ID = i.VINHO_ID
    )
GROUP BY
    p.nome_vinicola, v.nome, i.quantidade;
```

7. “V1. Crie uma view (vinhos_produtores) que apresente todos os vinhos dos produtores com os seguintes campos: nome do produtor, nome do vinho, tipo de vinho, teor alcoólico do vinho, região do vinho e ano da edição.”

```
CREATE VIEW vinhos_produtores AS
SELECT p.nome_vinicola AS Produtor, v.nome AS Vinho, v.tipo AS Tipo,
v.teor_alcoolico AS Teor_Alcoolico,
    r.denominacao AS Região, e.ano AS Ano
FROM produtor p
JOIN vinho v ON v.PRODUTOR_ID = p.PRODUTOR_ID
JOIN regioao r ON r.REGIAO_ID = v.REGIAO_ID
JOIN edicao e ON e.VINHO_ID = v.VINHO_ID
ORDER BY Produtor;
```

8. “V2. Crie uma view (empresas_maiores_compras) que apresente as empresas ordenadas decendentemente pelo valor total das compras de vinho efetuadas, com os seguintes campos: numero, nome, nif e morada da empresa, o numero total de faturas e o volume de compras.”

```
CREATE VIEW empresas_maiores_compras AS
SELECT
    c.numero_cliente AS numero,
    c.nome_cliente AS nome,
    c.nif,
    c.morada,
    COUNT(f.num_fatura) AS total_faturas,
    SUM(f.valor_total) AS volume_compras
FROM
    cliente c
JOIN
    fatura f ON c.numero_cliente = f.CLIENTE_numero_cliente
WHERE
    c.tipo = 'Empresa'
GROUP BY
    c.numero_cliente, c.nome_cliente, c.nif, c.morada
ORDER BY
    volume_compras DESC;
```

Sistema *Web-based*

Por fim, foi solicitado no terceiro e último ponto do enunciado da Parte II do projeto "VINISYS" a elaboração de um sistema *Web-based* por forma a permitir a gestão de dados integrados na Base de Dados de forma intuitiva e eficiente por parte da administração da empresa.

Com vista a cumprir os objetivos delineados no parágrafo anterior, foi solicitada a elaboração de três funcionalidades: (i) W1 – Registo de Colheitas; (ii) W2 – Listagem de Vinhos e (iii) W3 – Gestão de Vendas. Para cada uma das funcionalidades enunciadas foram ainda indicadas (através de alíneas) algumas especificações que as funcionalidades em questão deverão implementar.

Por forma a responder ao solicitado, procedeu-se à elaboração do sistema *web-based*, cujos arquivos *backend* (ficheiros .php) e *frontend* (ficheiros .js, .html e .css) enviam-se em anexo ao presente relatório (pasta Sistema_Web-based_Projeto_Vinisis - Parte 2”).

Envia-se ainda em anexo ao presente relatório, conforme solicitado na plataforma moodle, um vídeo de apresentação/demonstração do protótipo do sistema *web-based* (Demonstracao_Sys_Web_Based_g7.mp4).

Conclusão

Em síntese, o presente relatório aborda o desenvolvimento da Parte II do projeto proposto no âmbito da UC de Bases de Dados, centrado na implementação de automatismos, pesquisa de dados e a elaboração de um sistema *web-based* por forma a permitir a adequada gestão da base de dados “VINISYS”.

Os automatismos incluíram a implementação de *triggers*, *stored procedures* e funções, destinados a otimizar a manipulação dos dados e garantir a integridade do sistema. As consultas e *views* desenvolvidas permitiram extrair informações relevantes sobre vinhas, vinhos e clientes.

O protótipo *web-based* incorporou funcionalidades-chave para a gestão eficiente dos processos da empresa, como o registo de colheitas, a listagem detalhada de vinhos e a gestão de vendas e clientes. A interface foi concebida com o propósito de ser intuitiva, garantindo acessibilidade e facilidade de uso por parte dos gestores do sistema.

A abordagem técnica adotada na elaboração do protótipo *web-based* anteriormente referido seguiu os princípios de boas práticas de modelação e implementação de bases de dados e sistemas *web*, visando a escalabilidade, a centralização de dados e a eficiência na gestão das operações. Dessa forma, o protótipo *web-based* elaborado apresenta-se, conforme se poderá verificar no vídeo de apresentação do mesmo, como uma ferramenta robusta e prática para os desafios de gestão no setor vinícola.